

GÉNÉRATION ALÉATOIRE DE MOT DE PASSE 2

Programmation C++ Linux



Lycée Diderot, Paris

BTS CIEL 1

TABLE DES MATIÈRES

Table des illustrations	2
Introduction.....	3
Génération de mot de passe	3
Diagrammes UML.....	3
Cas d'utilisation	3
Séquence	3
Prérequis	4
Condition	4
Utilisation des fonctions rand() et srand()	5
Messages à l'utilisateur	6
Travail à réaliser	6
Index.....	6

TABLE DES ILLUSTRATIONS

Figure 1: Diagramme des cas d'utilisation	3
Figure 2: Diagramme de séquence.....	4

INTRODUCTION

Dans le TP précédent, vous avez créé un programme permettant de générer un mot de passe à partir d'un alphabet que vous avez déterminé. Le problème est que le mot de passe généré n'est absolument pas aléatoire. Il va falloir y remédier.

GÉNÉRATION DE MOT DE PASSE

On revoit ici les diagrammes UML déjà vus précédemment, avec une actualisation du diagramme de séquence.

DIAGRAMMES UML

CAS D'UTILISATION

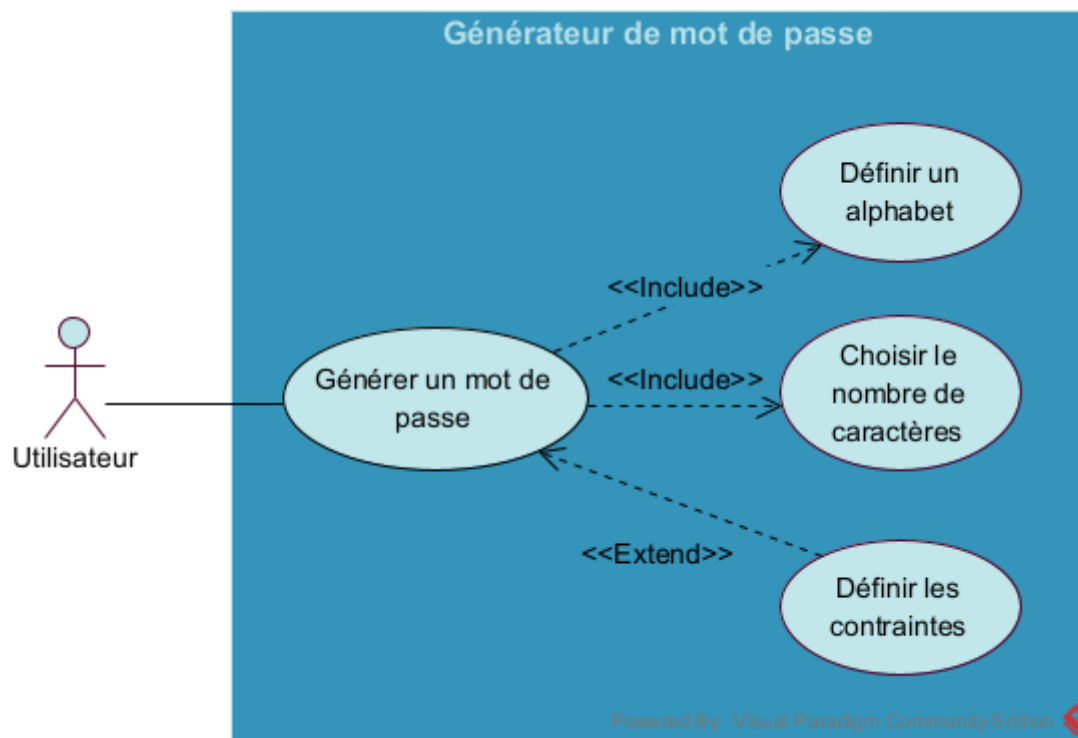


Figure 1: Diagramme des cas d'utilisation

La génération d'un mot de passe sous-entend que cela se fait de manière aléatoire: c'est l'objectif du jour.

SÉQUENCE

Un diagramme de séquence a, pour rappel, l'objectif de montrer l'interaction entre les différents objets d'un système. Ici, cela se complique un peu en ajoutant des alternatives.

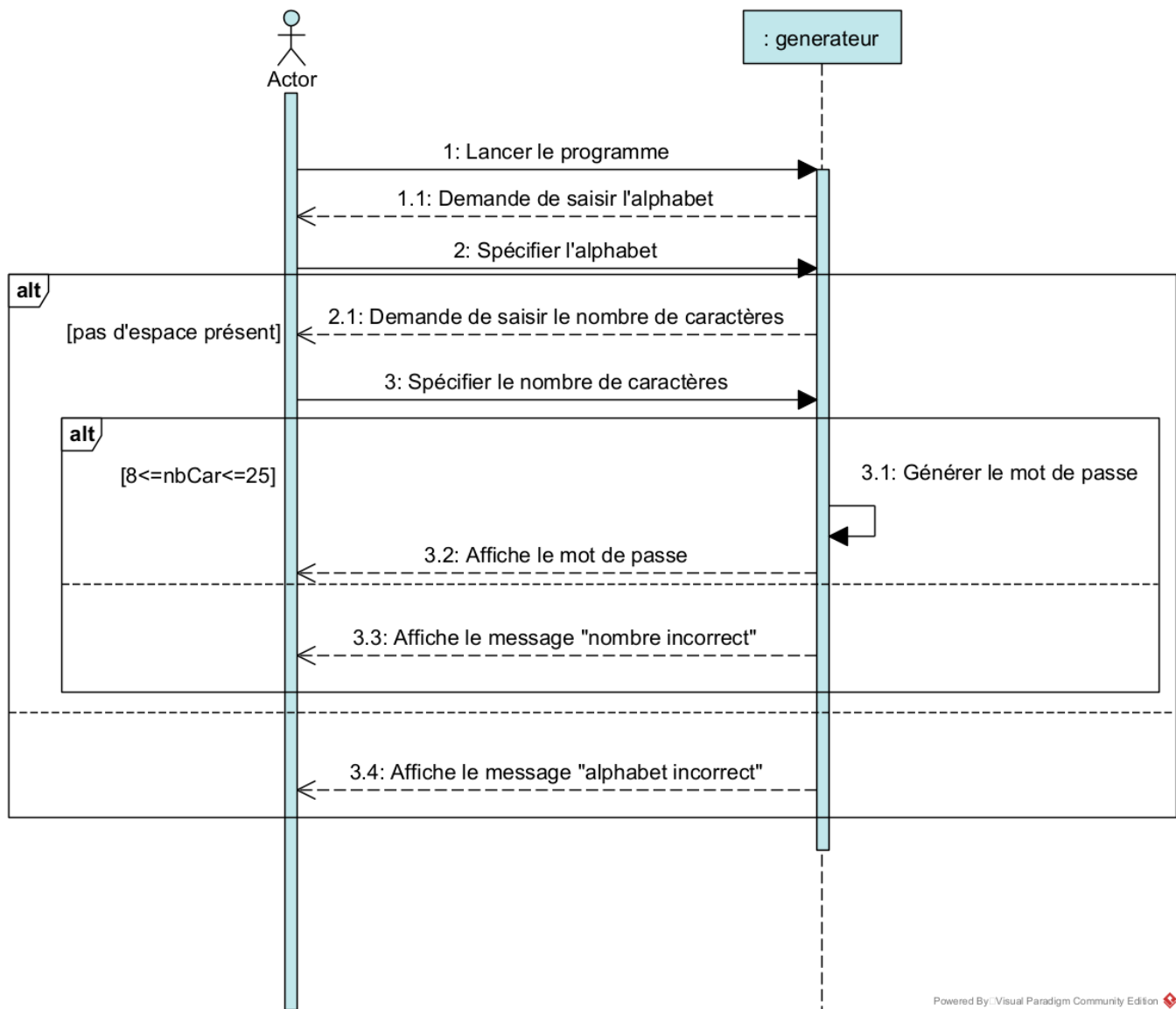


Figure 2: Diagramme de séquence

PRÉREQUIS

CONDITION

Dans le TP précédent, on n'a pas tenu compte du fait que l'utilisateur puisse limiter le nombre de caractères: on commence donc à définir le cas d'utilisation "Définir les contraintes".

On peut considérer aussi qu'un caractère espace dans l'alphabet n'est pas viable (il vous appartient de trouver comment saisir un espace, **cin** ne gère pas ce cas de figure)

Pour cela, il faut utiliser une structure conditionnelle: le si en algorithmique, **if** en **C++**.

Il s'écrit comme suit:

```
if (maCondition)
{
    instruction1;
    instruction2;
}
else
{
    instruction3;
    instruction4;
}
```

La condition d'un **if** est toujours un booléen (**bool**) c'est à dire soit vrai, soit faux. Les opérateurs de comparaison que l'on utilise sont: **!**, **<**, **>**, **<=**, **>=**, **==**, **!=**

L'objectif est d'afficher un message informant l'utilisateur s'il s'est trompé.

UTILISATION DES FONCTIONS **RAND()** ET **SRAND()**

La fonction **rand()** permet de générer un nombre aléatoire entre 0 et la constante **RAND_MAX**, soit 2147483647. C'est la théorie, en réalité, il s'agit d'une fonction pseudo-aléatoire. Elle donnera la même séquence de nombre à chaque fois que le programme sera lancé. Son algorithme a besoin d'un point de départ (**seed** ou graine) pour générer le nombre aléatoire. En changeant ce point, on peut donc changer le résultat. Si on se débrouille bien, on peut fixer le point de départ en fonction de l'horloge de l'ordinateur: c'est ce que permet la fonction **srand()** (le **s** est pour **seed**) combinée à la fonction **time()**.

Comment ça marche? Comme ceci:

```
srand(time(NULL)); //initialisation du point de départ
...
m_nombre = rand(); //appel de la fonction rand et
                  //récupération de la valeur aléatoire
                  //dans la variable m_nombre
```

Mais avant, il vous faut inclure une librairie de fonctions à votre fichier source afin de pouvoir utiliser ces fonctions. Vous devez utiliser **cstdlib**.

MESSAGES À L'UTILISATEUR

Votre programme doit communiquer avec l'utilisateur. Par souci de passer les tests Github, je vous donne les messages que votre programme doit afficher.

Numéro du message	Contenu
1.1	"Entrez un alphabet: "
2.1	"Entrez un nombre de caractères pour le mot de passe: "
3.3	"nombre incorrect"
3.4	"alphabet incorrect"

TRAVAIL À RÉALISER

Comme au TP précédent, la première chose à faire est la fiche de tests (ou de recette). Je vais là aussi vous en fournir une dans un fichier à part. Par contre, il se peut qu'elle soit incomplète. A vous de voir s'il vous faut ajouter des tests. De plus, certains attendus ne sont pas remplis, a vous de le faire.

Dans un souci de garder une trace de vos travaux précédents, il vous faut créer de nouveaux fichiers et ne pas réécrire sur les précédents.



ATTENTION

À chaque séance, vous devrez faire une sauvegarde de votre travail sur Github avec Github Desktop par un commit puis un push. Ne pas le faire vous vaudra un 0. Ne pas rendre de travail dans le temps imparti vous vaudra 0. Ne pas avoir la coche verte signifie que le code rendu ne satisfait pas tout à fait aux attendus mais ne signifie pas pour autant une mauvaise note. L'objectif est d'avoir la coche verte.

Ne pas m'appeler pour valider les différents exercices vous vaudra un 0.

INDEX

`rand()`, 4
`RAND_MAX`, 4

`srand()`, 4
`time()`, 4